MCP 표준의 이해

3-1. MCP란
3-2. 왜 표준이 필요한가
3-3. MCP의 구조
3-4. 쉬어가기: MCP는 왜 만든 회사가 공짜로 풀었나

3-1. MCP란

> MCP가 무엇을 표준화하는 규약인지 이해해봅니다.

MCP는 Model Context Protocol의 약자입니다. AI 애플리케이션과 외부 시스템(도구와 데이터)을 잇는 표준 규약을 말합니다. 2024년 11월 Anthropic이 발표한 개방형 표준입니다.

지금까지 만든 도구를 돌아보면 전부 LangChain 코드 안에 파이썬 함수로 존재합니다. 이 도구를 Claude Desktop이나 Cursor 같은 다른 AI 클라이언트에서 쓰려면 각 클라이언트의 형식으로 다시 만들어야 합니다.

MCP는 도구를 독립된 서버로 분리하고 통신 규약을 표준화합니다. 그래서 **한 번 만든 도구를 MCP를 지원하는 모든 클라이언트가 쓸 수 있게** 됩니다.

익숙한 것에 빗대면 USB가 PC와 주변기기 사이의 연결을 표준 하나로 통일한 것과 같습니다. Java 경험이 있다면 DB 벤더마다 접속 코드를 따로 짜는 대신 JDBC라는 표준 하나로 통일한 것을 떠올리면 됩니다.

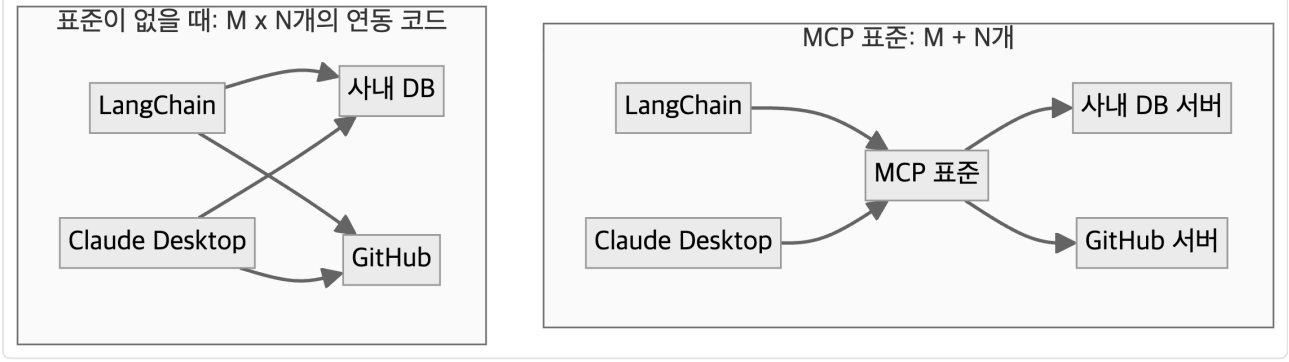
> MCP는 도구를 특정 프레임워크에서 분리해 표준 규약으로 노출하는 방식이라는 걸 확인했습니다.

3-2. 왜 표준이 필요한가

> 도구를 Agent마다 다시 만들지 않기 위한 표준의 가치를 이해해봅니다.

AI 클라이언트가 M개이고 연결할 시스템이 N개라고 해 봅니다. 표준이 없으면 필요한 연동 코드는 M 곱하기 N개입니다. 같은 사내 DB 하나를 위해 LangChain용, Claude Desktop용, Cursor용 도구 코드를 각각 만들어야 합니다.

표준이 있으면 서버 N개와 클라이언트 M개만 있으면 됩니다. 연동 코드가 M 더하기 N으로 줄어듭니다.



회사 관점에서 의미는 세 가지입니다.

- 이미 공개된 서버(filesystem, github, slack, postgres 등)는 그냥 가져다 씁니다. 도구 코드를 한 줄도 안 쓰고 능력을 추가할 수 있습니다.
- 사내 시스템을 MCP 서버로 만들어 두면 다른 팀과 다른 AI 클라이언트도 재사용합니다.
- 도구 정의가 특정 프레임워크에서 분리되므로 나중에 LangChain에서 다른 프레임워크로 옮겨도 도구는 그대로입니다.

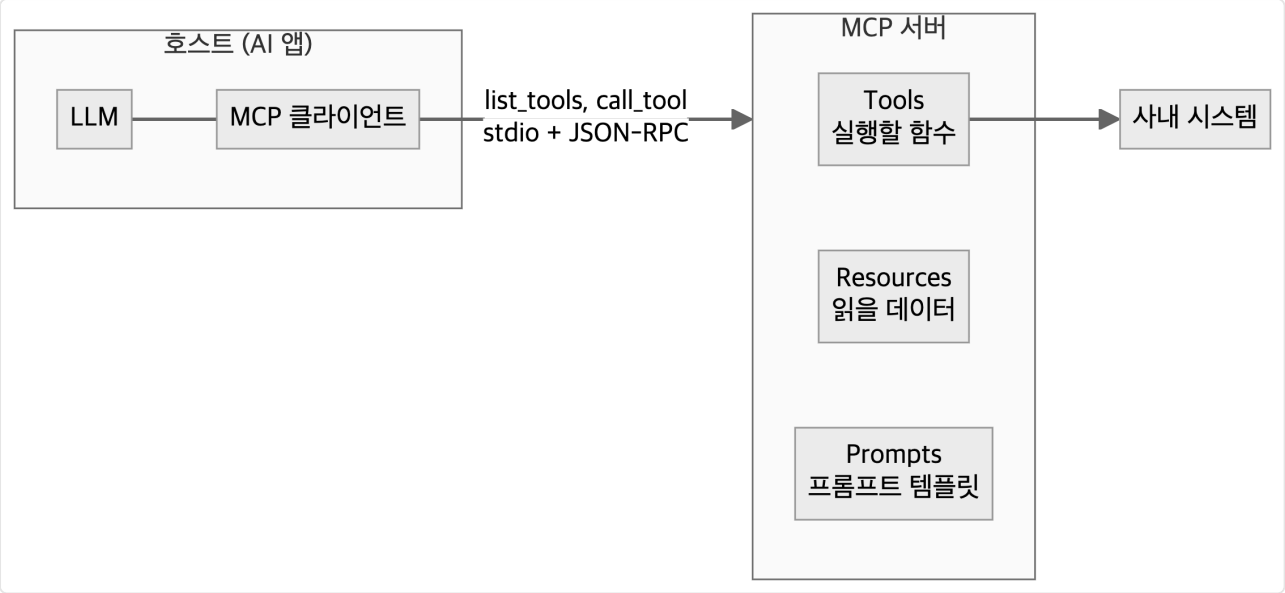
오늘 오후에 만들 MCP 서버가 정확히 이 그림의 서버 자리에 들어갑니다. 그리고 이 서버는 6일차에서 Multi-Agent의 부품으로 다시 쓰입니다.

> 표준이 M 곱하기 N의 연동 코드를 M 더하기 N으로 줄인다는 걸 확인했습니다.

3-3. MCP의 구조

> 서버, 클라이언트, 전송 방식 세 구성 요소를 정리해봅니다.

MCP의 구조는 다음과 같습니다.



1) 서버: 도구와 데이터를 노출하는 쪽입니다. 서버가 제공할 수 있는 것은 세 종류입니다.

종류	무엇	누가 부르나	예시
Tools	실행할 함수	LLM이 판단해서 호출	search_employee()
Resources	읽을 데이터 (URI 기반)	클라이언트가 URI로 접근	employees://1
Prompts	미리 정의된 프롬프트 템플릿	사용자가 명령으로 선택	/asset-report

실무에서 압도적으로 많이 쓰는 것은 Tools입니다. Resources와 Prompts는 4번째 세션 4-3 추가로 해볼 만한 것에서 맛만 봅니다.

2) 클라이언트: 호스트(AI 앱) 안에 들어 있는 연결 담당입니다. 서버에 "도구 목록 주세요"(list_tools)와 "이 도구 실행해 주세요"(call_tool) 같은 요청을 보냅니다.
이번 실습에서는 langchain-mcp-adapters의 MultiServerMCPClient가 이 역할을 합니다.

3) 전송 방식(transport): 클라이언트와 서버가 메시지를 주고받는 통로입니다.

방식	동작	장점	언제 쓰나
stdio	클라이언트가 서버를 자식 프로세스로 띄우고 표준 입출력으로 통신	가장 단순, 빠름, 외부 노출 없음	같은 머신. 이번 실습 기본
SSE	HTTP 기반 단방향 스트리밍	원격 가능	구형 서버 호환용, 신규 권장 아님
Streamable HTTP	HTTP 기반 양방향 스트리밍	원격 가능, 현행 권장 표준	원격 서비스, 운영 배포

결정 기준은 간단합니다. 같은 머신에서 돌리면 stdio이고 네트워크 너머라면 Streamable HTTP입니다. 이번 실습은 전부 stdio를 씁니다.

두 가지를 미리 짚어 둡니다. "서버"라는 단어 때문에 항상 떠 있는 웹 서버를 상상하기 쉽습니다. 다만 stdio 방식에서는 클라이언트가 필요할 때 서버 프로세스를 직접 띄우고 끝나면 함께 내려갑니다. 이때 포트는 열리지 않습니다.

그리고 MCP는 프레임워크가 아니라 프로토콜입니다. 특정 언어나 프레임워크에 묶이지 않은 메시지 규약이고 JSON-RPC를 씁니다. 다음 세션에서 쓸 fastmcp는 그 규약을 편하게 구현해 주는 파이썬 라이브러리 중 하나일 뿐입니다.

> 서버, 클라이언트, 전송 방식이라는 세 구성 요소와 stdio를 쓰는 기준을 익혔습니다.

3-4. 쉬어가기: MCP는 왜 만든 회사가 공짜로 풀었나

> 프로토콜을 오픈소스로 푼 판단이 왜 이득이었는지 봅니다.

2024년 11월 Anthropic이 MCP를 공개하면서 명세와 SDK를 통째로 오픈소스로 냈습니다. 자사 모델 전용으로 잠글 수도 있었는데 그러지 않았습니다.

3-2에서 본 M 곱하기 N 문제가 만든 쪽 발목도 똑같이 잡고 있었기 때문입니다. 도구 연동이 클라이언트마다 제각각이면 어떤 AI 제품도 사내 시스템에 붙는다고 약속하지 못합니다.

표준을 독점하는 쪽보다 표준이 존재하는 쪽이 이득인 구조였습니다. MCP 서버가 늘어날수록 그 서버를 쓸 수 있는 클라이언트의 값어치도 같이 올라갑니다.

2025년엔 다른 주요 벤더들도 각자 제품에 MCP 지원을 넣었습니다. 한 회사가 만든 규격이 벤더 중립 표준으로 자리 잡은 건 흔한 일이 아닙니다.

실무로 옮기면 이렇습니다. 사내 시스템을 MCP 서버로 한 번 만들어 두면 위에 얹는 AI 클라이언트가 바뀌어도 도구는 남습니다.
오늘 만드는 서버가 6일차 Multi-Agent에서 코드 한 줄 안 고치고 다시 쓰이는 게 그 축소판입니다.

- 명세와 공개 서버 목록은 modelcontextprotocol.io에 있습니다.

> 표준이 생기는 쪽이 이득이라 프로토콜이 공개됐고, 그 덕에 도구가 클라이언트보다 오래 남는다는 걸 확인했습니다.